

Prove CDuce

Davide Camino

February 8, 2025

Abstract

Ho fatto alcune prove cercando di riprodurre gli errori ottenuti da Lorenzo. Per ridurre la complessità ho cercato di usare ontologie molto semplici e programmi semplicissimi in modo da poter confrontare facilmente diverse versioni del software.

1 Set Sperimentale

Avendo usato un set sperimentale molto semplice posso riportare qui per intero tutto il codice sia dei programmi in CDuce che dell'ontologia che ho usato; questo secondo me è interessante perché permette di mostrare le differenze tra le varie versioni in dettaglio, e evidenziare come a seconda della versione una certa venga interpretato con diversi tag.

1.1 Codice

```
1 namespace owl="http://www.w3.org/2002/07/owl#"
2 namespace xml="http://www.w3.org/XML/1998/namespace"
3 namespace xsd="http://www.w3.org/2001/XMLSchema#"
4 namespace rdfs="http://www.w3.org/2000/01/rdf-schema#"
5 namespace skos="http://www.w3.org/2004/02/skos/core#"
6 namespace rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#";
7
8
9 let poeople = load_xml "persone.rdf";
10
11 type Ontology = <rdf:RDF xml:base=String> [ AnyXml* ];;
12
13 let ont :? Ontology = poeople;;
14
```

Il codice è molto semplice, da linea 1 a 7 si definiscono i namespaces, si carica un foglio XML (riga 9) si definisce la struttura di un'ontologia semplicissima specificando solo il nodo radice che ha come figli nodi di tipo `AnyXml` (riga 11) infine si valuta se il tipo di `people` è proprio `Ontology`.

1.2 Ontologia

```
1 <?xml version="1.0"?>
2 <rdf:RDF xmlns="http://www.semanticweb.org/dadi/people/"
3   xml:base="http://www.semanticweb.org/dadi/people/"
4   xmlns:owl="http://www.w3.org/2002/07/owl#"
5   xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
6   xmlns:xml="http://www.w3.org/XML/1998/namespace"
7   xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
8   xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
9 >
10 <owl:Ontology rdf:about="http://www.semanticweb.org/dadi/people"/>
11
12 <owl:Class rdf:about="http://www.semanticweb.org/dadi/people#Cani"/>
13
14 <owl:Class rdf:about="http://www.semanticweb.org/dadi/people#People"/>
15
16 <owl:NamedIndividual rdf:about="http://www.semanticweb.org/dadi/people#fido">
17   <rdf:type rdf:resource="http://www.semanticweb.org/dadi/people#Cani"/>
18 </owl:NamedIndividual>
19
20 <owl:NamedIndividual rdf:about="http://www.semanticweb.org/dadi/people#gianni">
21   <rdf:type rdf:resource="http://www.semanticweb.org/dadi/people#People"/>
22 </owl:NamedIndividual>
23
24 <owl:NamedIndividual rdf:about="http://www.semanticweb.org/dadi/people#mario">
25   <rdf:type rdf:resource="http://www.semanticweb.org/dadi/people#People"/>
26 </owl:NamedIndividual>
27
28 <owl:NamedIndividual rdf:about="http://www.semanticweb.org/dadi/people#pluto">
29   <rdf:type rdf:resource="http://www.semanticweb.org/dadi/people#Cani"/>
30 </owl:NamedIndividual>
31 </rdf:RDF>
32
```

Anche l'ontologia è davvero ridotta all'osso, abbiamo due classi che descrivono persone e cani, per ciascuna classe 2 individui, rispettivamente Gianni e Mario per le persone e Fido e Pluto per i cani.

1.3 Versioni di CDuce

Per gli esperimenti ho usato 3 differenti versioni di CDuce:

Ufficiale è l'ultima versione stabile; i problemi di questa sono quelli noti e scritti nella tesi, in particolare non siamo mai riusciti ad interfacciarci con Ocaml inoltre il processo di compilazione come suggerito dal sito ci è stato sconsigliato da Kim stesso

Repo Kim ho provato ad usare esattamente la stessa versione che Kim ha usato per costruire gli esempi che ci ha mandato

Attuale dev l'ultima versione di cui è stato fatto il commit del branch dev.

2 Esperimenti

2.1 Versione repo Kim

Comincio con questa versione perché ho dovuto scartarla subito, infatti quando provavo ad eseguire la riga 11 (dove si definisce la struttura dell'ontologia) senza aver eseguito la riga 7 veniva scatenato un errore interno.

```
1
2 >type Ontology = <rdf:RDF xml:base=String> [ AnyXml* ];;
3
4 Characters 17-24: Runtime error:
5   Internal error: Failure("Todo: 39")
6 Backtrace:
7
```

Ciò che ci aspettiamo invece è che se proviamo ad definire un elemento senza aver definito il namespace l'errore sia appunto qualcosa di simile a `Undefined namespace`.

Questo, il fatto che il codice di errore sia `"Todo: 39"` e non ci sia nessuna backtrace mi fa pensare, come giustamente suggerito da Lorenzo, che questa feature sia ancora da implementare.

2.2 Versione attuale dev

In questa versione eseguendo la linea 11 senza aver definito i namespace otteniamo l'errore che ci aspettiamo:

```
1
2 > type Ontology = <rdf:RDF xml:base=String> [ AnyXml* ];;
3
4 Characters 17-24: Runtime error:
5   Undefined namespace prefix rdf
6
```

Questo è un buon inizio, ho provato allora ad eseguire la riga 11 dopo aver definito il namespace e il comando è stato accettato correttamente.

Adesso, prima di provare a caricare i dati in formato XML, proviamo a costruire un elemento di tipo `Ontology` direttamente in `CDuce`; tradotto in codice significa:

```
1
2 namespace rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#";;
3 type Ontology = <rdf:RDF xml:base=String> [ AnyXml* ];;
4 let myOnt = <rdf:RDF xml:base="la mia ontologia"> [ ];;
5 let ont :? Ontology = myOnt;;
6
```

Anche l'esecuzione di questo breve pezzo va a buon fine.

Siamo ora pronti a definire tutti i namespace e a caricare il file XML. Il risultato è il seguente:

```
1
2 > let poeople = load_xml "persone.rdf";;
3
4 val poeople : AnyXml = <ns1:RDF
5   base="http://www.semanticweb.org/dadi/people/"
6   rdfs="http://www.w3.org/2000/01/rdf-schema#"
7   xml="http://www.w3.org/XML/1998/namespace"
8   rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
9   xsd="http://www.w3.org/2001/XMLSchema#"
10  owl="http://www.w3.org/2002/07/owl#">[
11    <ns1:Ontology about="http://www.semanticweb.org/dadi/people">[ ]
12    <ns1:Class about="http://www.semanticweb.org/dadi/people#Cani">[ ]
13    <ns1:Class about="http://www.semanticweb.org/dadi/people#People">[ ]
14    <ns1:NamedIndividual
15      about="http://www.semanticweb.org/dadi/people#fido">[
16      <ns1:type resource="http://www.semanticweb.org/dadi/people#Cani">[ ]
17      ]
18    <ns1:NamedIndividual
19      about="http://www.semanticweb.org/dadi/people#gianni">[
20      <ns1:type resource="http://www.semanticweb.org/dadi/people#People">[ ]
21      ]
22    <ns1:NamedIndividual
23      about="http://www.semanticweb.org/dadi/people#mario">[
24      <ns1:type resource="http://www.semanticweb.org/dadi/people#People">[ ]
25      ]
26    <ns1:NamedIndividual
27      about="http://www.semanticweb.org/dadi/people#pluto">[
28      <ns1:type resource="http://www.semanticweb.org/dadi/people#Cani">[ ]
29      ]
30    ]
31  ]
32
```

Qui sorgono i problemi, questo file infatti non può essere parsificato, dato che compaiono di nuovo i namespace segnato `ns1`. Questi compaiono sia che siano stati definiti i namespace sia che non lo siano stati. Il problema più grave però non è la presenza dei namespaces segnato, ma il fatto che lo stesso namespaces segnato faccia riferimento a differenti namespaces.

Guardiamo riga 14 e 16 e confrontiamole con le righe 16 e 17 dell'ontologia. I namespaces diversi `owl` e `rdf` dell'ontologia sono stati entrambi convertiti in `ns1` rendendoli a tutti gli effetti indistinguibili. Una cosa del genere non può essere parsificata.

2.3 Versione Stabile

Proviamo ora la versione stabile, quella che ho usato per scrivere la tesi. Qui ci aspettiamo che tutto funzioni correttamente, ma facciamo questo test per evidenziare le differenze tra quelli che sono i messaggi che ci aspettiamo di ricevere e quelli

che invece si ricevono con le nuove versioni. Nuovamente proviamo a definire la struttura dell'ontologia (riga 7 codice pagina 1) senza aver definito i namespaces, e come per la versione dev riceviamo un messaggio di errore coerente.

Proiamo allora a caricare il file XML e cerchiamo le differenze tra il risultato di questo caricamento e del precedente.

```
1
2 > let poeople = load_xml "persone.rdf";;
3
4 val poeople : AnyXml = <rdf:RDF
5   xml:base="http://www.semanticweb.org/dadi/people/">[
6     <owl:Ontology
7       rdf:about="http://www.semanticweb.org/dadi/people">[
8       ]
9     <owl:Class
10      rdf:about="http://www.semanticweb.org/dadi/people#Cani">[
11      ]
12     <owl:Class
13      rdf:about="http://www.semanticweb.org/dadi/people#People">[
14      ]
15     <owl:NamedIndividual
16      rdf:about="http://www.semanticweb.org/dadi/people#fido">[
17       <rdf:type
18         rdf:resource="http://www.semanticweb.org/dadi/people#Cani">[
19         ]
20       ]
21     <owl:NamedIndividual
22      rdf:about="http://www.semanticweb.org/dadi/people#gianni">[
23       <rdf:type
24         rdf:resource="http://www.semanticweb.org/dadi/people#People">[
25         ]
26       ]
27     <owl:NamedIndividual
28      rdf:about="http://www.semanticweb.org/dadi/people#mario">[
29       <rdf:type
30         rdf:resource="http://www.semanticweb.org/dadi/people#People">[
31         ]
32       ]
33     <owl:NamedIndividual
34      rdf:about="http://www.semanticweb.org/dadi/people#pluto">[
35       <rdf:type
36         rdf:resource="http://www.semanticweb.org/dadi/people#Cani">[
37         ]
38       ]
39   ]
40
```

Come possiamo vedere qui tutti i namespace sono riconosciuti correttamente, a prescindere che siano o meno stati dichiarati preventivamente. Questo è dovuto al fatto che CDuce è in grado di leggerli dallo schema XML riportato all'inizio dell'ontologia (righe da 5 a 9 del codice a pag 2).

Inoltre dato che queste informazioni riguardano lo schema XML e non i dati del foglio e di conseguenza non vengono stampati, vengono invece riportati nell'output della versione dev, infatti se guardiamo le righe da 6 a 9 dell'output di pagina 4 queste sono assenti (giustamente) dall'output della versione stabile.

Ovviamente in questa versione possiamo eseguire tutto il codice riportato a pagina 1 senza scatenare errori e il documento XML viene parsificato correttamente come un elemento di tipo [Ontology](#).

3 Conclusioni

Ho fatto altri test “off-camera” ma nulla è servito, le nuove versioni non sembrano riconoscere affatto i namespaces e non sono riuscito in alcun modo a caricare un documento senza i `ns1` e di conseguenza non sono riuscito a parsificare l'XML con le nuove versioni.

Il fatto che dalla versione usata da Kim per gli esempi quella attuale alcuni errori sono stati corretti mi fa pensare che sia solo questione di tempo prima che si possa usare la versione dev per i nostri esperimenti. Fino ad allora purtroppo credo che saremo vincolati ad usare la versione stabile con tutti i suoi problemi.

Si può però provare ancora a collegare OCaml a CDuce seguendo le istruzioni sul repo di Kim che spiega secondo molto meglio come fare rispetto alla guida sul sito, inoltre c'è anche l'esempio con i Makefile.